工程实践报告

面向班级协作的门户与截止日聚合系统设计与实现

ClassHub：一个低依赖、可离线、可安装的班级信息平台

项目名称：ClassHub 班级门户

课程方向：集成电路设计与集成系统

项目组：微电子学院 集成电路设计与集成系统 26-1 班

报告人：颜鋆宇

项目仓库：https://github.com/smallblackultra/classhub

报告日期：2026 年 11 月

摘 要

班级场景中的信息长期分散在 QQ 群、学习通、学院网站和临时表格里：通知会被聊天记录淹没，截止时间靠记忆，表格填报缺少名单状态，班委也无法确认“谁还没看、谁还没交”。本文设计并实现 ClassHub——一套面向大学班级的门户与截止日聚合系统。系统以 Python 3.12 标准库构建单体 HTTP 服务，以 SQLite（WAL）按班分库保存数据，前端采用 Preact/htm 与原生 JavaScript，无构建步骤。系统覆盖账号与权限、截止日/考试/表格的发布与完成状态、QQ 机器人与粘贴/书签导入、草稿审核、站内通知、完成报表、在线备份，以及班级门户、独立个人中心、学院新闻抓取。针对弱网和移动端使用，进一步实现根作用域 PWA、接口缓存、个人待办的离线队列与 client_key 幂等同步，并封装 Android WebView APK 与 Electron Windows 安装包。公网版本经 Cloudflare Tunnel、1Panel OpenResty 和 systemd 部署，8 个班级库可用；48 个单元/接口测试通过。结果表明，该方案在低依赖、低运维成本的前提下，能够把班级信息入口、截止日治理和离线可用性统一到一个可安装的 Web 应用中。

关键词：班级信息化；截止日管理；SQLite；PWA；离线优先；权限控制

Abstract

Class information is scattered across chat groups, learning platforms, college websites and ad-hoc spreadsheets. Notices are buried in chat history, deadlines rely on memory, and form collection lacks visibility into who has submitted. This report presents ClassHub, a class portal and deadline aggregation system. The backend is a single-process Python 3.12 standard-library HTTP service with per-class SQLite databases in WAL mode; the frontend uses Preact/htm and vanilla JavaScript without a build step. It provides authentication and role-based access, deadline/exam/form management, QQ-bot and paste/bookmark ingestion, draft review, notifications, reports, online backup, a class portal, a standalone personal center, and college news scraping. For weak networks and mobile use, a root-scope PWA caches GET responses and supports offline creation/update/deletion of personal plans, synchronized idempotently through a client_key. Android WebView and Electron wrappers

are also provided. The public deployment uses Cloudflare Tunnel, 1Panel OpenResty and systemd; eight class databases are served and 48 tests pass. The result shows that a low-dependency, low-maintenance architecture can unify class entry points, deadline governance and offline usability in an installable web app.

Keywords: class information system; deadline management; SQLite; PWA; offline-first; access control

目 录

1 绪论.....4
1.1 背景与问题.....4
1.2 目标与范围.....4
1.3 相关工作与取舍.....5
2 需求分析.....6
2.1 用户角色.....6
2.2 功能需求.....6
2.3 非功能需求.....6
2.4 约束条件.....7
3 系统设计.....8
3.1 总体架构.....8
3.2 数据模型.....8
3.3 权限与会话.....9
3.4 路由与部署拓扑.....9
3.5 离线与 PWA.....10
3.6 客户端封装.....10
4 关键实现.....11
4.1 服务端与路由.....11
4.2 DDL 与任务周历.....11
4.3 门户与个人中心.....12
4.4 新闻抓取.....13
4.5 离线缓存与同步.....13
4.6 原生壳.....14
5 测试与验证.....15
5.1 测试策略.....15
5.2 功能测试.....15
5.3 兼容性与离线验证.....16
5.4 安全与隐私.....16
5.5 结果.....16
6 部署与运维.....17
6.1 远端拓扑.....17
6.2 发布流程.....17
6.3 备份与恢复.....17
6.4 故障与降级.....18
7 总结与展望.....19
7.1 总结.....19
7.2 不足.....19
7.3 展望.....19
参考文献.....20
附录 A 主要接口.....21
附录 B 运行与测试.....21
附录 C 术语.....21

1 绪论

1.1 背景与问题

大学班级的信息流具有“多源、短时、强截止”的特点。作业和考试来自课程平台，表格和统计来自学院通知，临时事项来自班委在 QQ 群里的提醒。信息不是不存在，而是散落在不同介质中：群消息会被刷走，学习通的任务不会覆盖班级表格，学院官网的通知需要主动访问。对同学而言，真正重要的是“我这周还有什么没做”；对班委而言，则是“谁还没看到、谁还没交、哪些事项需要催”。这两类需求并不能被一个普通的待办清单同时满足。

本项目从班级真实使用场景出发，目标不是替代学校教务系统，而是做一个低门槛的“班级入口”：同学打开一个页面就能看到班级事项、学院新闻和个人待办；班委可以在同一套权限体系下发布、审核、统计和备份；在外网和手机端也能通过 HTTPS 访问。系统还必须足够轻：班级没有专职运维，部署不应该依赖复杂的前端构建、消息队列或云数据库。

1.2 目标与范围

系统由三部分组成：面向全班公开的班级门户（`/`）、面向班级成员的 DDL 应用（`/daliy/`）以及独立个人中心（`/personalcenter/`）。门户负责品牌展示、学院新闻和班级入口；DDL 应用负责截止日、考试、表格与完成状态；个人中心合并门户与 DDL 的账号、头像、个人待办、密码与班级工具。三者共享同一套账号、Cookie 与 API 权限。

在功能边界上，系统不替学校教务系统做排课，不实现学习通的账号代管，也不做面向全校的选课/成绩功能。它只处理班级可自行维护、可审核、可追踪的事项，并对未实现模块（归寝、活动报名、课程信息等）保持明确的占位状态，不用假数据冒充真实功能。

1.3 相关工作与取舍

常见方案包括通用待办应用、在线表格、课程平台自带提醒和班级群机器人。通用待办缺少班级受众与审核；在线表格缺少截止提醒和权限；课程平台不覆盖班级表格；群机器人只能触达，不能沉淀。ClassHub 的取舍是：用一个单体服务把“采集—审核—发布—完成—统计—备份”串起来，前端尽量无构建，数据落 SQLite，部署只依赖 Python 与 Nginx/OpenResty。这样做的代价是横向扩展能力有限，但对 8 个班、数百人规模的班级场景是足够的。

2 需求分析

2.1 用户角色

系统区分普通同学、班长/团支书、系统管理员与待审批账号。可见性由条目的 audience 决定，而不是由 category 决定；同学访问班委条目返回 404 而不是 403，避免通过状态码探测条目是否存在。角色与权限的对应关系见表 1。

表 1 角色与主要权限

能力	普通同学	班长/团支书	管理员
查看班级事项	是	是	是
查看班委事项	否（404）	是	是
新建/编辑/发布	否	是	是
审核草稿	否	是	是
成员审批/邀请码	否	是	是
全局管理员操作	否	否	是

2.2 功能需求

（1）账号与权限：注册后先进入 pending，班委审批后生效；登录支持跨班定位；首次登录强制改密；写操作需要 CSRF。（2）DDL 管理：支持作业、表格、任务、考试四类，支持班级/班委受众、指定参与人、草稿/发布/归档、完成与不适用、催填名单、修订历史和审计。（3）采集：粘贴文本、QQ 群 @机器人、学习通书签，AI 只做字段修正并生成草稿，任何情况下不自动发布。（4）通知与报表：截止前 24 小时/2 小时、逾期提醒、站内通知、完成报表与 CSV。（5）门户与个人中心：新闻双栏、个人待办、头像、密码、班级工具。（6）离线与客户端：PWA 安装、缓存只读数据、个人待办离线增删改、联网幂等同步；Android/Windows 壳。

2.3 非功能需求

低依赖：服务端只使用 Python 标准库，前端不引入构建链；可维护：模块、测试、文档与部署脚本同仓；安全：密码使用 scrypt，会话 Cookie 为 HttpOnly，写操

作校验 CSRF，接入令牌只存哈希；可用性：静态门户与 API 分离，新闻抓取失败时读缓存；响应式：1440×900、1280×800、390 宽移动端均不出现横向溢出；可恢复：8 个班库、portal.db 与 .secret 纳入备份。

2.4 约束条件

班级没有专职运维，部署需要在一台低配 Linux 云主机上完成；源站不直接暴露 80 端口，由 Cloudflare Tunnel 终止公网 TLS；1Panel 提供 OpenResty 与静态站点；SQLite 单文件不适合多进程写入，因此服务保持单进程；学院官网没有稳定 API，新闻抓取必须按 HTML 结构解析并准备降级。

3 系统设计

3.1 总体架构

系统采用“静态门户 + Python 单体服务 + SQLite + 边缘隧道”的架构，如图 1 所示。浏览器、Android WebView 和 Electron 壳都访问同一个 HTTPS 源；Cloudflare 负责 DNS、TLS 与 Tunnel；1Panel OpenResty 对 `/` 与`/personalcenter/` 直接服务静态文件，对 `/api/`、`/daliy/`、`/daily/` 反代到 systemd 管理的 Python 服务；Python 通过 sqlite3 访问 8 个班库与全局 portal.db；学院官网只作为新闻数据源。

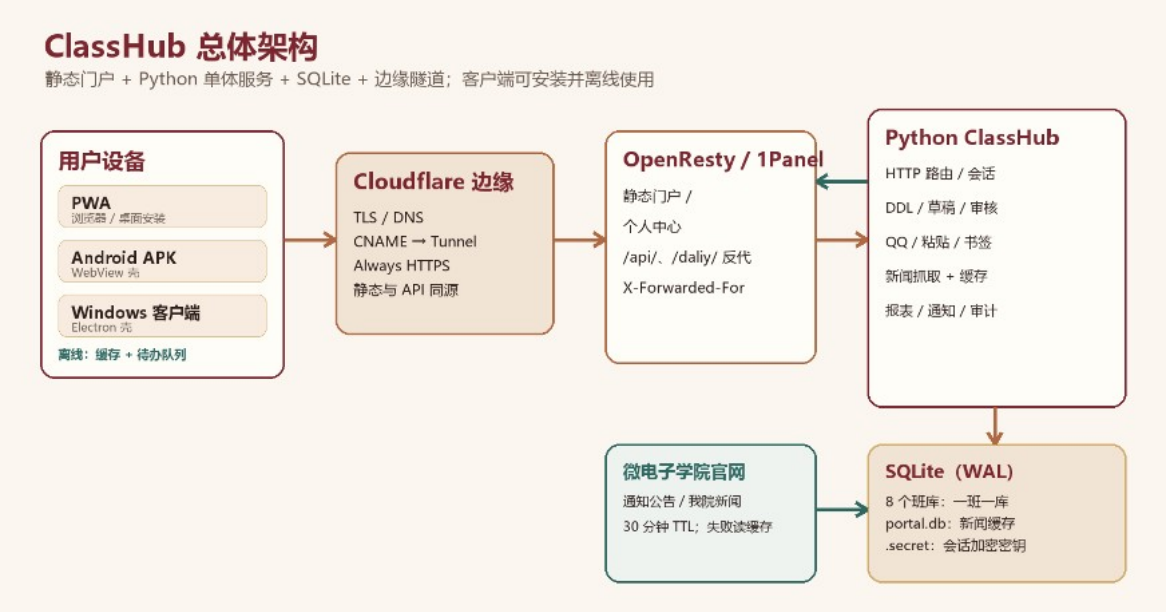


图 1 系统总体架构

这种拆分把“读静态页面”和“写业务数据”分开：门户即使 Python 暂时重启也能打开；DDL 与个人中心需要登录，统一走 API；新闻抓取失败时首页仍能显示缓存或空状态，不会拖垮首屏。

3.2 数据模型

每个班级拥有独立 SQLite 库，避免跨班数据耦合；portal.db 只存跨班共享的新闻与元数据。班库包含 14 张表，核心表及职责见表 2。所有时间统一存 UTC ISO8601 字符串，由应用层按 Asia/Shanghai 展示。

表 2 班库核心表

表	职责
classes / users	班级、账号、角色、状态、头像、强制改密
deadlines	事项主体、受众、类别、截止时间、来源、状态、去重键
completions / deadline_targets	完成/不适用、指定参与人
sources / raw_messages / extraction_jobs	接入源、原始消息、AI/规则抽取任务
revisions / audit_logs	事项修订历史与操作审计
notifications / personal_items	站内通知、个人待办（含 client_key）
sessions / settings	会话、CSRF、全局设置

personal_items 的 `client_key` 是离线同步的关键：它由客户端生成，服务端在同一用户范围内建立唯一部分索引；重复 POST 时先查已有行，若并发插入触发 IntegrityError 再回查，保证同一本地计划只产生一条服务端记录。

3.3 权限与会话

登录时先查当前班库，找不到再遍历 8 个班库，定位成功后将班级 slug 写入 `cdh_cls` Cookie；会话 id 存在 sessions 表，浏览器只持有 HttpOnly 的 `cdh_sid`。写操作要求 `X-CSRF` 与 `cdh_csrf` 双提交一致。路由层统一声明 auth、committee、csrf、allow_pending，业务层再按 audience 过滤，避免“页面看不到但接口能拿到”的漏洞。

3.4 路由与部署拓扑

Python 服务同时支持本地直接访问与反代前缀：`CDH_BASE` 默认 `/daily`，`APP_PREFIX` 为 `/daliy`，`APP_ALIASES` 包含 `/daily`。裸前缀会 302 到带斜杠路径；`/api` 先于静态资源匹配；根路径在非 app 模式返回 home.html，在 app 模式返回 index.html。远端由 OpenResty 直接服务静态根，因此根路径不会进入 Python，减少了无谓的进程压力。

3.5 离线与 PWA

PWA 采用根作用域：`/sw.js` 的 scope 为 `/`，同时覆盖首页、个人中心与 DDL；旧 `/daliy/sw.js` 注册被移除，避免嵌套作用域冲突。Service Worker 只缓存静态壳与导航回退，不缓存 `/api/`；API 缓存由页面层的 `offline.js` 负责，这样数据权限和“按学号隔离”由业务层控制，而不是由缓存层猜测。离线数据流见图 2。

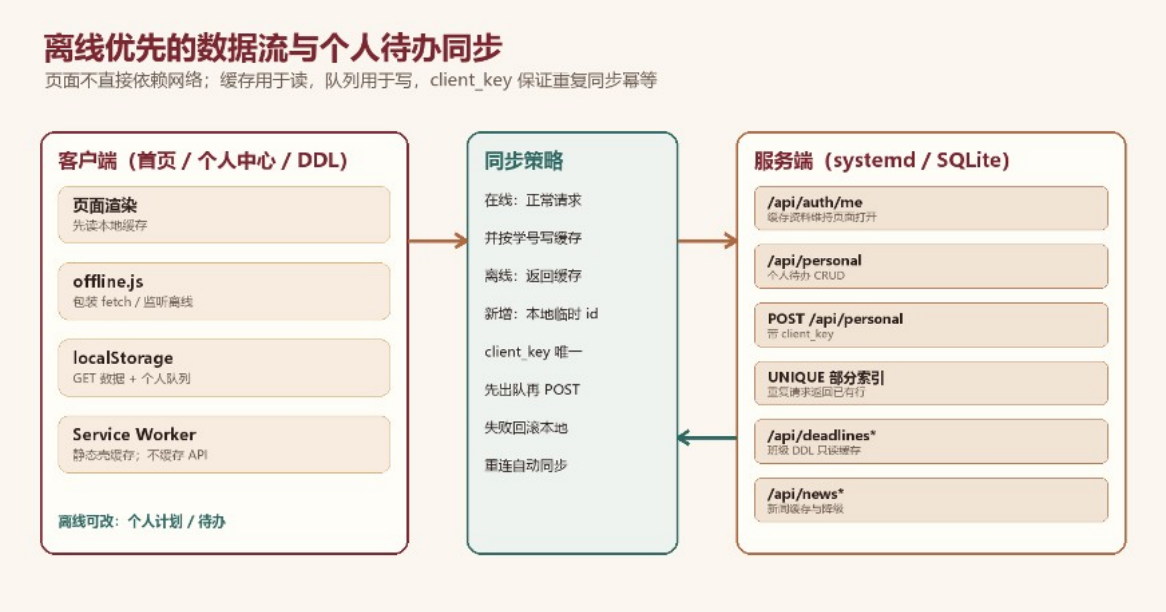


图 2 离线数据流与个人待办同步

离线读取时，成功的 GET JSON 按学号写入 localStorage；`/api/auth/me` 使用上次缓存资料，使页面在没有网络时仍能进入个人中心或 DDL。离线写入只允许个人计划/待办：新增项生成临时 id 与 client_key，修改/删除写入队列；联网后 `syncPersonal()` 先出队再请求，失败回滚，服务端以唯一索引兜底。班级 DDL 与新闻保持只读缓存，避免离线产生无法审核的班级数据。

3.6 客户端封装

Android 端使用 WebView 加载公网 HTTPS 入口，设置 INTERNET 与 ACCESS_NETWORK_STATE 权限，minSdk 24、target 35，图标使用学院院徽；Windows 端使用 Electron + electron-builder 生成 NSIS 安装包，安装到用户目录，提供桌面/开始菜单快捷方式。两端都复用同一 PWA 与账号体系，原生壳只负责窗口、图标与安装体验，不复制业务逻辑。

4 关键实现

4.1 服务端与路由

服务端基于 `ThreadingHTTPServer`，每个请求解析 cookie、会话与 CSRF 后进入路由表。路由装饰器把权限声明与处理函数放在一起，避免权限检查散落在各分支；静态资源与 API 在同一个 handler 中分流，API 响应统一 no-store，静态资源使用 no-cache 以配合版本化查询参数。错误以 JSON 返回，前端 API 层统一把 401、403、409 映射为登录、权限与冲突提示。

4.2 DDL 与任务周历

DDL 列表支持“全部/作业/任务/考试”筛选、搜索、按截止/最新排序，以及任务周历。周历以周一为起点，把每个任务画成从本周一（或创建日）延伸到截止日的横向圆角条：跨周任务延伸到周日并标注实际截止日期，按截止日排序，最多展示 8 条；点击日头或任务条可只查看该日任务，再次点击取消。桌面端一屏内可见 7 天与任务条，移动端保留 7 列并在卡片内部横向滚动，避免压缩到不可读。图 3 为 DDL 主界面。



图 3 DDL 主界面与任务周历（演示数据）

4.3 门户与个人中心

门户是公开静态页：顶部为院徽、导航与个人中心入口，主视觉使用校园航拍背景与「厚德·笃行 / 崇实·尚新」渐变标题；右侧新闻双栏；下方为归寝、DDL、活动三张服务卡；底部为每日轮换的名言与日期。页面在 1440×900、1280×800 等 16:10 视口内单屏展示，移动端改为纵向流。图 4、图 5 分别为门户和独立个人中心。

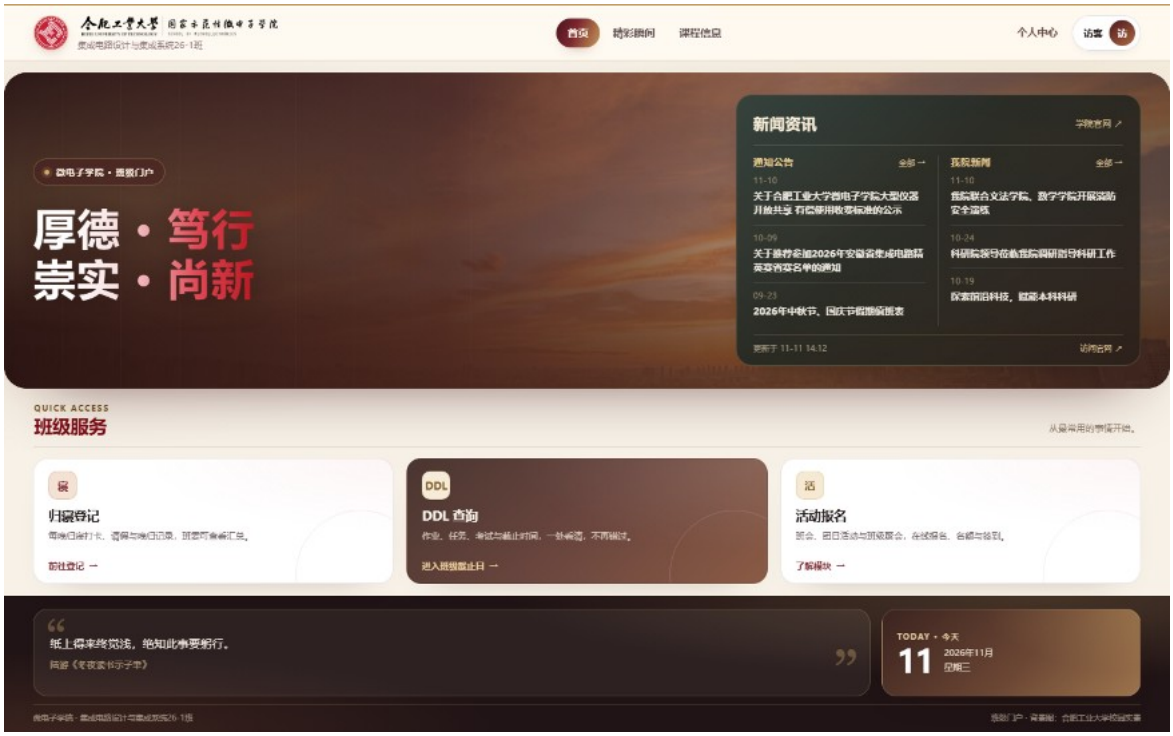


图 4 班级门户首页（1440×900）

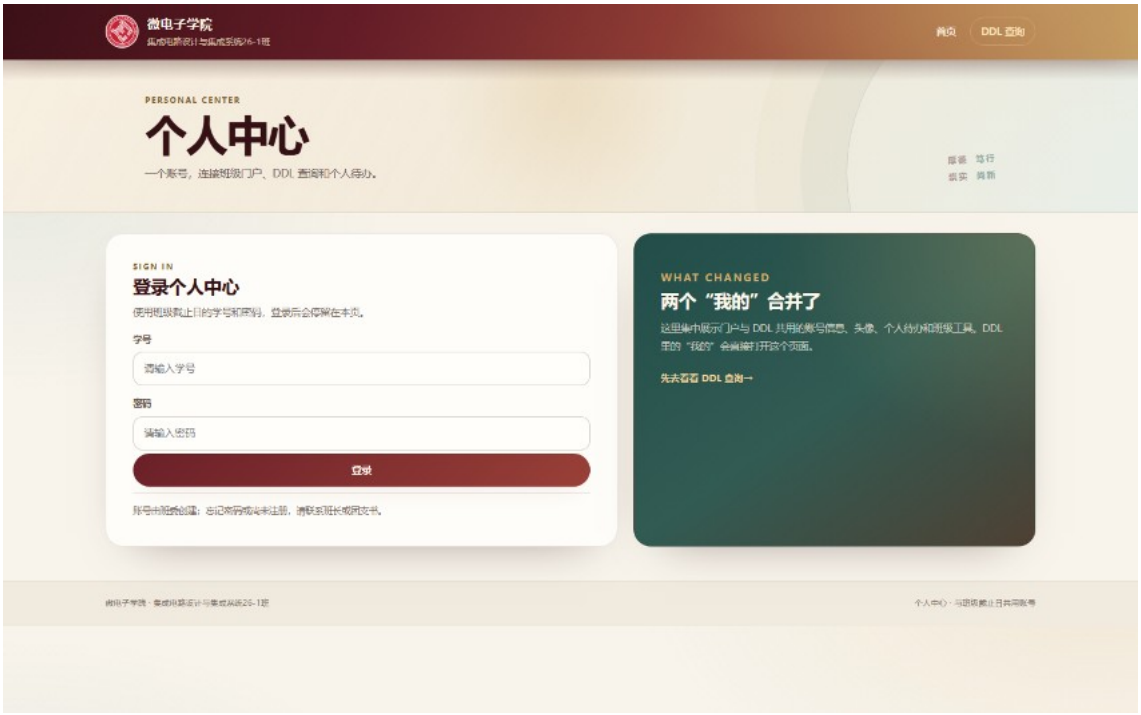


图 5 独立个人中心（演示账号，未登录态）

个人中心不另建账号体系：它调用同一组 `/api/auth/*` 与 `/api/personal*`，提供登录、资料、头像上传/清除、个人待办、班级工具、改密与退出。首页头像只展示，不响应点击；DDL 的「个人中心」入口直接跳转 `/personalcenter/`，避免用户误入 DDL 后再被重定向。登录/改密页与 DDL 顶栏都提供「返回首页」。

4.4 新闻抓取

新闻来自微电子学院官网的通知公告列表与我院新闻列表。解析器只允许 `wdzxy.hfut.edu.cn` 的 https 链接，按 `li.news`、日期节点、标题与摘要提取，按发布时间倒序排序；置顶的旧公告不会被误当成最新。抓取结果写入 portal.db，TTL 默认 30 分钟；刷新失败时保留旧缓存并标记 stale，页面对应显示“缓存/更新时间”，不伪造新闻。

4.5 离线缓存与同步

`offline.js` 在页面最早阶段包装 `fetch`：在线 GET 成功后按学号缓存 JSON；离线时从缓存构造 Response；对 `/api/personal` 的 GET 会把服务端数据与本地队列合并，保证离线新增/修改/删除立即可见。同步顺序为“创建 → 修改 → 删除”，

每个本地项带 client_key；服务端唯一索引是最后一道幂等保障。图 6 为移动端门户，离线时同一套页面从缓存打开。



图 6 移动端门户（390 宽，离线缓存可打开）

4.6 原生壳

Android 项目使用 Gradle 构建 debug 包，路径含中文时通过 `android.overridePathCheck=true` 规避 Gradle 限制；APK 使用 debug 签名，包名 com.hfut.classhub，图标为院徽。Windows 端用 electron-builder 生成 NSIS 安装包；为避开无管理员权限下的符号链接失败，关闭 `signAndEditExecutable`，再用本地缓存的 rcedit 写入 exe 图标，最后从已解包目录重新打包安装器，保证安装器与安装后的 exe 都显示院徽。

5 测试与验证

5.1 测试策略

测试以 Python 标准库 unittest 为主，启动真实 HTTP 服务与临时 SQLite 库，覆盖路由、鉴权、CSRF、越权、数据校验、幂等和降级；前端通过 Node 语法检查、浏览器截图与 Service Worker 状态检查验证。测试账号全部为演示数据，不读取生产班库，报告与仓库均不包含真实学生信息。

5.2 功能测试

表 3 主要测试范围与结果

范围	代表用例	结果
门户/路由/PWA	根页面、资源、302、SPA 回退、manifest/SW	通过
认证与权限	登录、CSRF、待审批、班级/班委可见性	通过
DDL 与完成	CRUD、冲突、完成/不适用、报表、CSV	通过
接入与审核	QQ 令牌/幂等、粘贴、书签、未解析队列	通过
新闻/头像/待办	解析排序、降级、头像格式、client_key 幂等	通过
离线/PWA	根 SW、壳缓存、离线页、待办队列	通过（手工离线）

全量命令 `python -X utf8 -m unittest discover -s tests -t . -v` 共 48 项通过；Node `--check` 覆盖首页、个人中心、DDL 与周历脚本。离线测试在 Chrome DevTools 中模拟离线，验证首页、个人中心、DDL 从缓存打开，离线新增待办后恢复网络只产生一条服务端记录。

5.3 兼容性与离线验证

桌面端在 1440×900、1280×800 和 1920×1200 检查，门户单屏不出现纵向滚动；移动端用 390×844 设备模拟检查，文档 scrollWidth 为 390，无横向溢出；DDL 周历在移动端内部横向滚动。PWA 在全新 Chrome 配置下注册根作用域 SW，确认 cache 为 classhub-shell-v28；正常加载后断网刷新仍显示缓存内容。Electron 打包版加载公网页面并离线刷新通过。

5.4 安全与隐私

密码使用 scrypt 加盐哈希；管理员查看密码依赖 `.secret` 密钥，仓库不保存该文件；接入令牌只存 SHA-256；登录、改密、审核与导出等关键动作写审计；AI 与 QQ 机器人只产草稿，没有绕过审核直接发布的路径；生产 HTTP 只监听 127.0.0.1，由边缘终止 TLS。公开仓库版本移除了生产主机、真实姓名、班级账号清单、初始口令规则与运维记录，只保留演示数据、架构说明和脱敏后的部署文档。

5.5 结果

系统已完成从“采集—审核—发布—完成—统计—备份”的闭环，并在公网稳定运行；门户、个人中心、DDL、新闻、PWA、Android/Windows 壳均可访问。当前未实现的归寝/活动/课程模块以占位页明确标注，不误导用户。48 项测试和多次浏览器检查覆盖主要路径与边界，满足班级规模的使用要求。

6 部署与运维

6.1 远端拓扑

公网入口为 `https://www.2026hfutintegratedcircuit1.com/`，Cloudflare Tunnel 将流量送到 1Panel OpenResty：静态门户与个人中心由 OpenResty 直接服务，`/api/`、`/daliy/`、`/daily/` 反代到 systemd 管理的 Python 服务。服务监听 127.0.0.1:8000，数据目录 `/opt/classhub/data`，8 个班库 + portal.db + .secret；每日 03:17 备份。组件与职责见表 4。

表 4 远端组件

组件	职责
Cloudflare	DNS、TLS、Tunnel、Always HTTPS
1Panel OpenResty	静态站点、反向代理、缓存控制
systemd classhub	Python 常驻、开机自启、环境变量
SQLite（8 班库 + portal.db）	班级数据、新闻缓存、个人待办
cron 备份	每日打包班库/portal.db/.secret，保留 14 天

6.2 发布流程

静态资源采用“备份旧目录 → 解包新目录 → 校验关键标记 → 清 Cloudflare 缓存”的流程；Python 代码按 release 目录解包，切换 current 软链接后重启 classhub。发布前先跑全量测试与 Node 检查；发布后验证 `/`、`/personalcenter/`、`/daliy/`、`/api/health`、`/api/news` 与 SW 版本。前端版本通过 `?v=` 参数和 SW 版本号双重控制，避免用户被旧缓存锁住。

6.3 备份与恢复

在线备份调用 `python -m app --backup`，生成 8 个班库与 portal.db 的快照；离线脚本 `backup.ps1` 再把当前运行产生的数据库、`.secret` 与代码打包为 zip。恢复时先停服务、解包到数据目录、确认 `.secret` 权限为 0600、再启动并检查 `PRAGMA integrity_check`。备份不进入 Git 仓库，公开仓库只保留脚本与说明。

6.4 故障与降级

新闻抓取失败时读旧缓存并显示 stale；Python 重启时门户静态页仍可用；API 不可用时个人中心显示离线提示并读本地缓存；离线个人待办同步失败保留队列；数据库写冲突以 409 提示刷新；DDL 条目不可见时统一 404，避免泄露条目存在性。已知限制包括单进程 SQLite、无推送、新闻依赖 HTML 结构、离线只读班级数据、原生包未正式签名。

7 总结与展望

7.1 总结

本文完成了一套面向班级的低依赖信息平台：以 Python 标准库和 SQLite 支撑业务，以静态门户降低首屏依赖，以同一套 API 连接门户、个人中心与 DDL，以 PWA/离线队列解决弱网和移动端，以 Android/Windows 壳提供安装入口。工程上坚持“先权限、再数据、后页面、最后测试文档”的顺序，未实现模块保持占位，不用演示数据冒充真实功能。

7.2 不足

系统仍是单进程 SQLite，无法支撑大规模并发写；新闻解析对官网 HTML 结构敏感；离线不支持班级事项的发布与审核；Android 仅 debug 签名、Windows 安装包未签名，正式分发需要证书与上架流程；没有邮件/企微推送，通知只到站内；公开仓库为保证隐私，省略了生产运维记录与真实名册。

7.3 展望

后续可把数据访问抽象为仓储层，平滑迁移到 PostgreSQL；为新闻抓取增加多源与结构化元数据；为离线增加加密与服务端合并策略；补齐归寝、活动报名、课程信息等模块；增加 PWA 分享、消息推送与班级数据导出；在获得正式签名后发布 Android/Windows 安装包。

参考文献

[1] Python Software Foundation. http.server — HTTP servers[EB/OL]. https://docs.python.org/3/library/http.server.html.

[2] SQLite. Write-Ahead Logging[EB/OL]. https://www.sqlite.org/wal.html.

[3] MDN Web Docs. Service Worker API[EB/OL]. https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API.

[4] web.dev. The Offline Cookbook[EB/OL]. https://web.dev/articles/offline-cookbook.

[5] Preact. Preact — 3kB alternative to React[EB/OL]. https://preactjs.com/.

[6] developit. htm — Hyperscript Tagged Markup[EB/OL]. https://github.com/developit/htm.

[7] Cloudflare. Cloudflare Tunnel[EB/OL]. https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/.

[8] 1Panel. 1Panel 文档[EB/OL]. https://1panel.cn/.

[9] Android Developers. WebView[EB/OL]. https://developer.android.com/develop/ui/views/layout/webapps/webview.

[10] Electron. Electron Documentation[EB/OL]. https://www.electronjs.org/docs/latest/.

[11] 合肥工业大学微电子学院. 学院新闻与通知公告[EB/OL]. https://wdzxy.hfut.edu.cn/.

[12] Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases[EB/OL]. https://docs.python.org/3/library/sqlite3.html.

附录 A 主要接口

表 A1 主要公开/登录接口

方法	路径	说明
GET	/api/health	健康检查
GET	/api/meta	班级与分类元数据
POST	/api/auth/login	登录，设置 HttpOnly 会话
GET	/api/auth/me	当前用户与班级资料
POST	/api/auth/avatar	上传/清除头像
GET/POST	/api/personal	个人待办列表/创建
PATCH/DELETE	/api/personal/{id}	修改/删除个人待办
GET	/api/deadlines	班级事项列表
GET	/api/news	学院新闻双栏数据

附录 B 运行与测试

本机运行：在项目根目录执行 `python -m app --reset`（仅演示库）后访问 `http://127.0.0.1:8000/`。测试：`python -X utf8 -m unittest discover -s tests -t . -v`，共 48 项。生产运行：`python -X utf8 -m app --host 127.0.0.1 --port 8000 --no-notify`，由 systemd 托管。Android 构建：`gradle assembleDebug`；Windows 构建：`electron-builder --win nsis`。

附录 C 术语

表 C1 术语表

术语	含义
DDL	Deadline，本文指作业/考试/表格/班级任务等截止事项
PWA	Progressive Web App，可安装、可离线的 Web 应用
SW	Service Worker，浏览器后台脚本，负责静态壳缓存与导航回退

术语	含义
WAL	SQLite Write-Ahead Logging，提高并发读与写入稳定性
client_key	客户端生成的幂等键，用于离线待办重复同步去重